CODE COMPREHENSION

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

GUIDELINE 1

Frame Reading Around Concrete Questions, Not Linear Scans

Guideline 1: Frame Reading Around Concrete Questions

- Before opening any file, write 3 to 5 specific questions you need answered.
- Read code only to answer those questions.
- Stop reading when the questions are answered, or reformulate them when stuck.

Reasoning

- Comprehension is fact finding, not understanding the whole system
- Nearly all comprehension activity is driven by a specific information need
- Reading code without a question wastes time on lines unrelated to the task

Key Practices

- Write each question in plain English before opening any file
- When a question is answered, write a one-sentence answer and move to the next one
- If unanswered after 10 minutes, reformulate the question

GUIDELINE 2

Switch Deliberately Between Top-Down and Bottom-Up Reading

Guideline 2: Switch Deliberately Between Top-Down and Bottom-Up

- Start top-down: form a hypothesis from the function name and signature, then read the body to confirm or refute it.
- When your hypothesis breaks, switch to bottom-up: trace individual statements and data flow.
- When re-grounded, lift back up to a new hypothesis. Treat the switch as a deliberate move.

Reasoning

- Program comprehension can be hypothesis-driven and top-down
- Expert programmers also build a bottom-up 'program model' from individual statements
- Real developers switch between both modes based on familiarity
- Key lesson: notice which mode you are in and switch intentionally when you get stuck

Key Practices

- Before reading the body, write a one-sentence guess based only on the function name, parameters, and return type
- Read the body to confirm or refute that guess; if confirmed, you are done
- If refuted, go bottom-up: pick one statement, identify every variable and where it came from
- Once re-grounded, write a new one-sentence hypothesis
- Confusion usually means you are stuck in one mode; switch deliberately

GUIDELINE 3

Name What Each Variable and Helper Function Is For Before Tracing the Body

Guideline 3: Name Variable Roles Before Tracing

- For every variable, write a short label for what it is for over the function's lifetime.
- Use standard role names: fixed value, stepper, most-recent holder, most-wanted holder, gatherer, follower, one-way flag, walker, container, or temporary.
- Summarize each helper function in one sentence before tracing any calling code.

Reasoning

- About 9 to 10 role labels cover roughly 99% of all variables in procedural programs
- Students taught with role labels produced summaries closer to expert comprehenders
- Roles are what experienced developers tacitly apply when reading; naming them makes the move explicit
- Cannot assign a role to a variable? That variable is the bottleneck for comprehension
- Same logic applies to function calls: no one-sentence summary means your trace is guesswork

Key Practices

- For each parameter and local variable, write one short label: what is this variable for over the lifetime of the function?
- Prefer the 10 standard role names; invent a phrase only when none of them fits
- For each helper the code calls, write a one-sentence summary of what it returns or mutates
- If you cannot write that sentence for a helper, jump to the helper and resolve it first
- Only after all variables and calls are labeled, start tracing the body

References (1/2)

- [1] Brooks, R. (1983). Towards a theory of the comprehension of computer programs. Int'l Journal of Man-Machine Studies, 18(6), 543-554.
- [2] Pennington, N. (1987). Stimulus structures and mental representations in expert comprehension. Cognitive Psychology, 19(3), 295-341.
- [3] Von Mayrhauser, A. & Vans, A. M. (1995). Program Comprehension During Software Maintenance. Computer, 28(8), 44-55.
- [4] Sajaniemi, J. (2002). An empirical analysis of roles of variables in novice-level programs. IEEE HCC '02, 37-39.
- [5] Robillard, M. P., Coelho, W. & Murphy, G. C. (2004). How effective developers investigate source code. IEEE TSE, 30(12), 889-903.
- [6] Sajaniemi, J. & Kuittinen, M. (2005). An experiment on using roles of variables in teaching. Computer Science Education, 15(1), 59-82.

References (2/2)

- [7] Sillito, J., Murphy, G. C. & De Volder, K. (2006). Questions programmers ask during software evolution tasks. FSE '06, 23-34.
- [8] LaToza, T. D., Garlan, D., Herbsleb, J. D. & Myers, B. A. (2007). Program comprehension as fact finding. ESEC/FSE 2007, 361-370.
- [9] Ko, A. J., DeLine, R. & Venolia, G. (2007). Information Needs in Collocated Software Development Teams. ICSE '07, 344-353.
- [10] LaToza, T. D. & Myers, B. A. (2010). Hard-to-answer questions about code. PLATEAU '10, Article 8.
- [11] Dagenais, B. et al. (2010). Moving into a new software project landscape. ICSE '10, 275-284.
- [12] Maalej, W., Tiarks, R., Roehm, T. & Koschke, R. (2014). On the Comprehension of Program Comprehension. TOSEM, 23(4), Article 31.